Agentic Foundation Models and Memory Systems

Tool Use, MCP, Memory, Multi-Agent Systems, and Agent Evaluation

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 11, 2026

Table of Contents

1. Motivation
2. Learning Outcomes
3. Agent Architectures and Planning
4. Tool Learning and Tool-Use Evaluation
5. Tool and Context Interoperability: MCP
6. Memory Systems
7. Multi-Agent Collaboration
8. World Models and Environment Interaction
9. Agent Evaluation and Benchmarking
10. Agent Safety and Operations
11. Summary

The Agent Is No Longer a Prompt

- In 2023 an agent was a *prompt scaffold* wrapped around a chat model. ReWOO, LATS, multi-agent debate and Toolformer are now history lessons.
- By 2026, agency is **trained into the model** and **engineered around it**:
  - **Trained behaviour.** Kimi K2 learns from 3,000+ real MCP tools and 20,000+ synthetic ones. DeepSeek-V3.2 trains in 1,800+ environments.
  - **Planning inside the model.** Every major API now asks the client to pass reasoning state back across tool calls.
  - **An engineered harness.** Context, memory, permissions and sandboxes are designed, not prompted.

Claude Opus 4.6 finishes tasks of about **12 hours** at 50% success, but only about **70 minutes** at 80%. Capability is ahead of reliability.

Kimi Team, "Kimi K2: Open Agentic Intelligence," arXiv:2507.20534. DeepSeek-AI, "DeepSeek-V3.2," arXiv:2512.02556. METR, Time Horizon 1.1 data file (suite updated 8 May 2026).

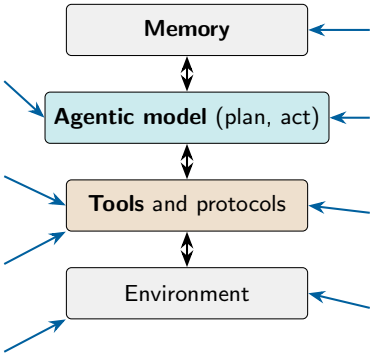
Today's Map: One Agent, Eight Questions

1. What came after ReAct?
Architectures, planning, trained agency

2. Does it call tools correctly?
Tool learning, training, evaluation

3. Who is trusted, and for what?
MCP, permissions, security

6. Can it rehearse before acting?
World models and environments



4. What should it remember?
Memory types, stores, retrieval hybrids

5. When does a second agent help?
Multi-agent patterns, costs, failures

7. How do we measure it?
Benchmarks, reliability, time horizons

8. How do we run it safely?
Identity, audit, cost, incidents

Learning Outcomes

By the end of this session, you should be able to:

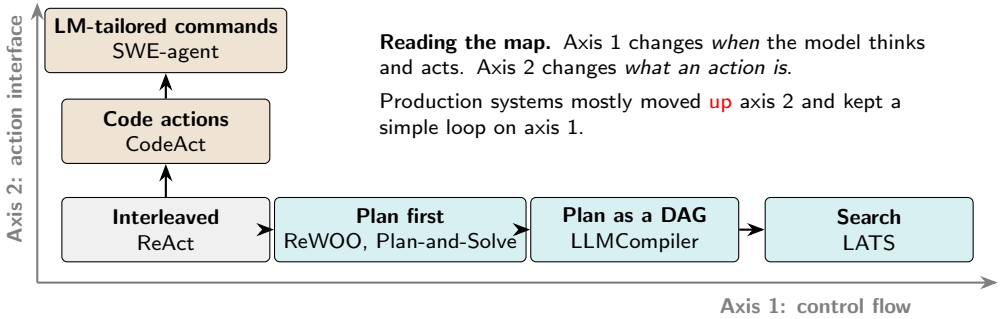
- **Explain** how agents moved from plan-first scaffolds to RL-trained agentic models.
- **Analyse** tool use: schema, selection, arguments, execution feedback and recovery.
- **Describe** the 2026 Model Context Protocol, its trust boundaries and its attacks.
- **Compare** memory architectures by memory type, write policy and retrieval structure.
- **Decide** when a multi-agent system pays for its coordination cost.
- **Evaluate** when a world model or a synthetic environment helps an agent.
- **Report** an agent benchmark score with the conditions that give it meaning.
- **Design** the operational layer: identity, approval, audit trails and cost controls.

Prerequisite knowledge: the ReAct loop and function calling; MCP roles and primitives; RAG, GraphRAG, HippoRAG and Self-Route; tree search with RAP, PlanBench and LLM-Modulo; self-consistency and best-of-N; outcome-reward RL; pass@k and LLM judges; prompt injection, least privilege, sandboxing, approval gates and CaMeL; model-based RL.

Agentic Models: **Architectures and Planning**

ReAct gave us the loop. What did the next three years add to it, and what did they remove?

Two Axes Beyond ReAct



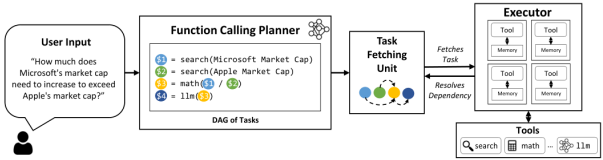
Reading the map. Axis 1 changes *when* the model thinks and acts. Axis 2 changes *what an action is*.

Production systems mostly moved up axis 2 and kept a simple loop on axis 1.

- JSON tool calls sit at the origin: one call, one observation, repeat.
- This section walks axis 1 first, then axis 2, then asks where planning went.

Synthesis of Xu et al., arXiv:2305.18323. Kim et al., arXiv:2312.04511. Zhou et al., arXiv:2310.04406. Wang et al., arXiv:2402.01030. Yang et al., arXiv:2405.15793.

Plan First: ReWOO and LLMCompiler



- **ReWOO.** ReAct re-sends its history at every tool call. Plan once instead, with variables that the tool outputs fill in later. HotpotQA: **1,986 tokens** against 9,795, at similar accuracy.
- **LLMCompiler** (above). The plan is a DAG, executed in parallel: up to 3.7× faster and 6.7× cheaper than ReAct.

Historical numbers. gpt-3.5-era results on short QA. Prompt caching has since weakened the token argument.

Xu et al., "ReWOO: Decoupling Reasoning from Observations," arXiv:2305.18323, Table 2. Figure: Kim et al., "An LLM Compiler for Parallel Function Calling," arXiv:2312.04511, Figure 2.

Tree Search Is Expensive

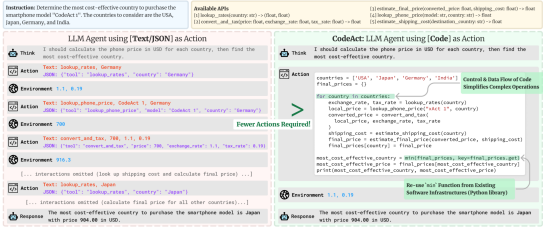
- **LATS** runs tree search over whole agent trajectories. It needs an environment that can be reverted, so that a failed branch leaves no trace.
- **The cost check.** Kapoor et al. re-ran these agents and tracked the bill. A "warming" baseline, which retries at rising temperature, beat both Reflexion and LATS.
- LATS cost about **55**× as much as the baseline, for lower accuracy.

HumanEval, GPT-4	Acc.	Cost
Simple "warming" baseline	93.2%	$2.45
Reflexion	87.8%	$3.90
LATS	88.0%	$134.50

In production the verifier is a **test suite, a linter or a screenshot**, not the model's own opinion.

Zhou et al., "Language Agent Tree Search," arXiv:2310.04406. Kapoor et al., "AI Agents That Matter," arXiv:2407.01502.

Axis 2: Code as the Action Space



- **CodeAct** (above) on M3ToolEval, 82 multi-tool tasks: **74.4%** success for code actions, against 52.4% for JSON and 53.7% for text, in 5.5 turns instead of 7.6.
- **Afterlife.** OpenHands ships a CodeAct agent. Presenting MCP servers as code APIs cut one Anthropic example from 150,000 to 2,000 tokens.

Wang et al., "Executable Code Actions Elicit Better LLM Agents," arXiv:2402.01030, Figure 1. Wang et al., "OpenHands," arXiv:2407.16741. Anthropic, "Code execution with MCP" (Nov 2025).

Axis 2: The Agent-Computer Interface

- **Thesis.** An LM agent is a new kind of end user. It needs its own interface, as humans need an IDE.
- **SWE-agent's design rules:**
  - simple, compact actions
  - informative but concise feedback
  - **guardrails**: a linter rejects broken edits
- A windowed file viewer and a capped search (at most 50 results) keep context small.
- Anthropic reports the same lesson: they spent more time on the tools than on the prompt.

Variant	SWE-bench Lite
Full interface	18.0%
No linting	15.0%
30-line window	14.3%
Full-file window	12.7%
Iterative search	12.0%
No edit command	10.3%

Same model, GPT-4 Turbo. Only the interface changes.

The absolute scores are from 2024. Open models now report 73 to 80% on SWE-bench Verified. The ablation is what lasts.

Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv:2405.15793, Table 3.
Anthropic, "Building Effective Agents" (Dec 2024).

Planning Moves Into the Model

Provider	Reasoning state	What the client must do
Anthropic	thinking blocks between tool calls	keep the blocks in the turn
OpenAI Responses	reasoning items	pass them back around function calls: about +3% on SWE-bench
Google Gemini	encrypted thought signatures	echo them back unchanged
Open models	`<think>` content	keep it in history. Removing it hurts (MiniMax-M2)

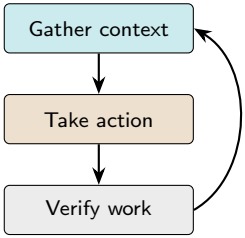
- The model now plans and replans *in its thinking, between tool calls*. Kimi K2 Thinking stays coherent over **200 to 300** consecutive calls.

Planning-first became a trained behaviour. The price is an API contract: **preserve the reasoning state**.

Anthropic, extended thinking docs. OpenAI, Responses API cookbook, "Reasoning items." Google, Gemini thinking docs. MiniMax-M2 and Kimi K2 Thinking model cards.

From Prompt Engineering to Context Engineering

- **Context rot.** Recall degrades as context grows. Treat context as a finite *attention budget*.
- **Four techniques:** compaction, structured notes, sub-agents that return 1 to 2 thousand token summaries, and just-in-time retrieval.
- **Across many windows.** An *initializer* agent writes a progress file and a list of 200+ features marked pass or fail. A *coding* agent then advances one feature per session.

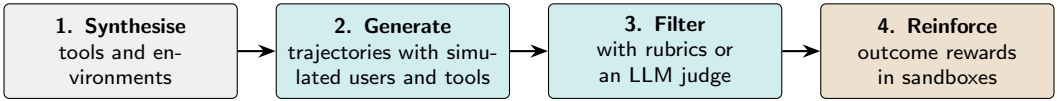


the Claude Agent SDK loop

The guidance shifted from *"which loop pattern?"* (2024) to *"how do we manage state across many context windows?"* (late 2025).

Anthropic: Rajasekaran et al., "Effective context engineering for AI agents" (Sep 2025). Young, "Effective harnesses for long-running agents" (Nov 2025). Shihipar, "Building agents with the Claude Agent SDK" (Sep 2025).

Agentic Foundation Models: The Open-Lab Recipe

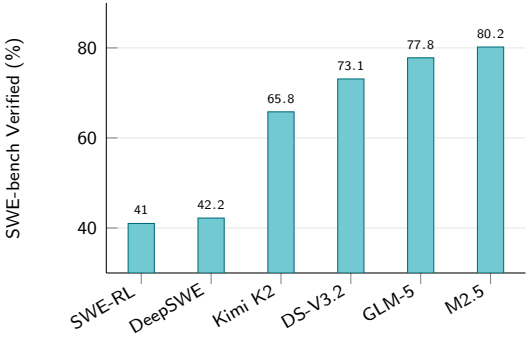


- **Kimi K2.** 3,000+ real MCP tools plus 20,000+ synthetic ones, an LLM user, a *stateful* tool simulator, and real sandboxes for code.
- **GLM-4.5.** Outcome rewards, with a format penalty that ends the trace when a tool call fails.
- **DeepSeek-V3.2.** 1,800+ environments and 85,000 prompts. **Qwen3-Coder.** 20,000 parallel environments.

The RL objectives are standard. What is new is the **scale of environments and verifiers**.

Recipe is a synthesis of: Kimi Team, arXiv:2507.20534. GLM-4.5 Team, arXiv:2508.06471. DeepSeek-AI, arXiv:2512.02556. Qwen, "Qwen3-Coder" blog (Jul 2025).

Trained for Agency: What the Reports Show



Open models, February 2025 to early 2026.

- **Open models** roughly doubled their score in a year.
- **Closed labs** disclose less. Deep research was trained with RL on browsing tasks. Operator used SFT for perception, then RL.
- **One disclosed reward.** codex-1 often claimed success on impossible tasks. Penalising results inconsistent with its actions raised honest "could not complete" reports from **0.15 to 0.85**.

The scaffolds differ across these reports, so this is **not** one controlled curve. Scores now also include "context management" in the harness.

Wei et al., arXiv:2502.18449. Together AI, DeepSWE blog. Kimi Team, arXiv:2507.20534. DeepSeek-AI, arXiv:2512.02556. GLM-5 and MiniMax-M2.5 model cards. OpenAI system cards: deep research, Operator, codex-1.

Does the Scaffold Still Matter?

Evidence	Finding	Reading
METR tasks (Feb 2026)	Opus 4.5 in Claude Code beats a plain ReAct scaffold in only 50.7% of bootstrap samples	scaffold barely matters
Terminal-Bench 2.0	GPT-5.3-Codex: 64.7 on Terminus 2, 77.3 on the Codex harness	a vendor-tuned harness adds 12.6 points

- The Terminal-Bench authors still conclude that *"model selection is usually more important than agent scaffold."*
- Yu Su: *"stronger LLMs may require less scaffolding (i.e., more 'reactive')"*.

METR, "Measuring time horizon using Claude Code and Codex" (Feb 2026). Merrill et al., "Terminal-Bench," arXiv:2601.11868. Google DeepMind, Gemini 3.1 Pro Model Evaluation (Feb 2026). Su, Berkeley CS294 (Feb 2025).

Architectures: What to Remember

- Plan-first scaffolds and tree search are the **2023 answers**. They cut tokens or raised accuracy, at a cost that simple baselines often beat.
- What lasted is the **action interface**: code actions and interfaces designed for the model.
- Planning moved **into the model** as interleaved thinking. The client must now carry reasoning state across tool calls.
- Frontier agency is **trained**: synthetic tools, simulated users, judged trajectories and RL in sandboxes.
- The harness is part of the system, and part of the score.

Next. Every one of these agents acts through tools. How does a model learn to call them, and how do we know it calls them correctly?

Agentic Models: **Tool Learning and Evaluation**

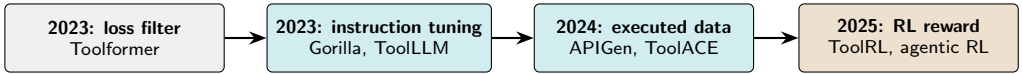
The model can emit a valid call. Does it pick the right tool, fill the right arguments, and recover when the call fails?

Learning to Use Tools: A Utility Filter

- **Toolformer (2023)** asked a self-supervised question: *does this API call make the following text easier to predict?*
- Let L_i^+ be the language-model loss after position i with the call and its result as a prefix, and L_i^- the loss without the result. Keep the call only if

$$L_i^- - L_i^+ \geq \tau_f$$

- Tool use emerged only at about **775M parameters**. The method could not chain tools or use them interactively.



The filter is a *utility reward* in disguise. It returns as execution-verified data, then as an RL reward.

Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv:2302.04761. Lineage is a synthesis with Liu et al., arXiv:2406.18518 and Qian et al., arXiv:2504.13958.

Scaling to Real APIs: Retrieval Enters the Loop

- **Gorilla.** Fine-tune with the retrieved API document *in the prompt*, so the model adapts when documentation changes.
- It introduced **AST matching**: a call is correct if its syntax tree matches a reference. A call to a tool that does not exist is a *hallucination*.
- **ToolLLM.** 16,464 real REST APIs. A depth-first search over calls (DFSDT) can backtrack, where ReAct follows one path: pass rate 64.8 against 40.2 for ChatGPT.
- The catch: a poor retriever can cut Gorilla's accuracy by up to 52%.

HuggingFace APIs	Accuracy	Halluc.
Gorilla (7B)	71.68	10.95
GPT-4	19.80	37.16
GPT-3.5	16.81	35.73
Claude	6.19	77.65

Zero-shot, 2023 models. A small tuned model beat frontier models that invented APIs.

Patil et al., "Gorilla: Large Language Model Connected with Massive APIs," arXiv:2305.15334, Table 1. Qin et al., "ToolLLM," arXiv:2307.16789, Table 4.

Autonomous Tool Creation: Models That Write Their Tools

- **LATM.** A strong model *makes* a tool once. A cheap model *uses* it many times.
  - *Propose* a function from 3 examples
  - *Verify* it with generated unit tests
  - *Wrap* it with usage demos
- A dispatcher decides whether to reuse a cached tool or make a new one.
- **CRAFT** abstracts verified solutions into a retrievable toolset. Removing the abstraction step drops GQA F1 from 48.8 to 39.7.

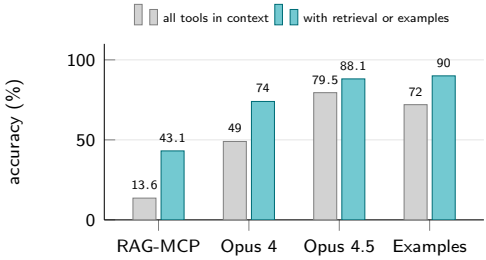
GPT-3.5 as user	CoT	LATM
Logical deduction	66.4	79.7
Tracking objects	61.6	99.6
Dyck language	20.4	92.2
Word sorting	59.2	98.3
Schedule meeting	18.9	100.0

These papers test **closed task families**, where verification is cheap. The open-world form today is a skill library plus retrieval.

Cai et al., "Large Language Models as Tool Makers," arXiv:2305.17126, Table 2. Yuan et al., "CRAFT," arXiv:2309.17428.

Tool Discovery: Too Many Tools Hurt

- **The cost.** Five MCP servers expose 58 tools. Their definitions take about **55K tokens** before the user says anything.
- **The damage.** In ReWOO, 17 of 20 new failures after going from 2 to 7 tools were tool misuse.
- **The fix is retrieval.** Keep tool schemas out of context until the model searches for them.
- **Programmatic calling.** Orchestrate tools from code, so intermediate results stay out of context: 37% fewer tokens.



Four separate evaluations. "Examples": example calls in the tool definition, on complex parameters.

A 2026 critique: with several valid tools per query, 30 to 47% of reported retriever gains are scoring artefacts.

Anthropic, "Introducing advanced tool use" (Nov 2025). Gan and Sun, "RAG-MCP," arXiv:2505.03275. Xu et al., arXiv:2305.18323. Zhu et al., "Tool Retrievers Are Underestimated," arXiv:2609.08327.

Training for Tool Use: From Data Engines to Rewards

- **Data engines (2024).** APIGen keeps a synthetic call only if it passes three checks: format, real execution, semantic review.
- **RL (2025).** ToolRL scores each call with a fine-grained reward:

$$R_{\text{final}} = R_{\text{format}} + R_{\text{correct}}, \qquad R_{\text{format}} \in \{0, 1\}, \quad R_{\text{correct}} \in [-3, 3]$$

  where R_{correct} combines tool-name, parameter-name and parameter-value matches.
- **Result.** Qwen2.5-7B rises from 41.97% to **58.38%** on BFCL with about 4K samples. Fine-grained rewards beat binary ones, and RL from a cold start beats SFT then RL.

Data *shrinks* as methods move to RL: 126K examples (ToolBench), 60K (APIGen), about 4K (ToolRL).

Liu et al., "APIGen," arXiv:2406.18518. Qian et al., "ToolRL: Reward is All Tool Learning Needs," arXiv:2504.13958.

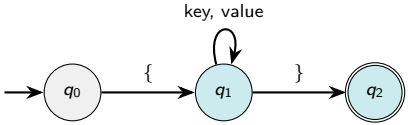
Schema Adherence: Constrained Decoding

- **Goal.** Make an invalid call *impossible to sample*.
- **Outlines.** Compile the format into an automaton, and index offline the tokens allowed in each state:

$$\sigma : Q \to \mathcal{P}(V)$$

Each step masks the logits to $\sigma(q_t)$, at $O(1)$ average cost.

- **JSON needs a grammar.** OpenAI compiles the schema to a context-free grammar. XGrammar makes this up to 100× faster.



In state q_1 only tokens in $\sigma(q_1)$ keep their logits. All others are set to $-\infty$.

This is why "strict" modes can promise valid output. The promise covers **syntax only**.

Willard and Louf, "Efficient Guided Generation for Large Language Models," arXiv:2307.09702. Dong et al., "XGrammar," arXiv:2411.15100. OpenAI, "Introducing Structured Outputs in the API" (Aug 2024).

Schema Adherence in Practice: Syntax Is Not Semantics

Schemas accepted and satisfied	LM only	Guidance	Outlines	XGrammar	OpenAI
GlaiveAI	0.90	0.96	0.95	0.93	0.89
GitHub, easy	0.65	0.86	0.59	0.79	0.29
GitHub, hard	0.13	0.41	0.03	0.28	0.09

- **Coverage is the hidden limit.** Closed APIs comply on the schemas they accept, but *reject* most hard ones.
- **Does constraining hurt reasoning?** One paper says yes. A re-run with matched prompts found no loss: GSM8K 0.77 free against 0.78 structured.
- **What no grammar can fix.** The right tool, and the right *values*.

Geng et al., "JSONSchemaBench," arXiv:2501.10868, Table 4 (values taken from a summarised extraction: verify before quoting). Tam et al., "Let Me Speak Freely?" arXiv:2408.02442. Kurt, "Say What You Mean" (.txt blog). OpenAI, Structured Outputs announcement (Aug 2024).

Reliability: pass@k Becomes pass^k

- Code benchmarks ask: does *at least one* of k samples pass? A customer-facing agent must pass *every* time.
- With c successes in n trials of a task:

$$\text{pass}^k = \mathbb{E}_{\text{task}}\left[\binom{c}{k} \Big/ \binom{n}{k}\right]$$

- τ-**bench** grades the final database state after a dialogue with a simulated user, under a policy document.

Setting	**1 trial**	**all k**
gpt-4o, τ retail (2024)	61.2	25 (k=8)
gpt-5-medium, MCPMark	52.6	33.9 (k=4)
Opus 5, τ^2 banking (2026)	48.7	32.0 (k=4)

The gap did not close as models improved. Harder domains reopened it.

User simulators are noisy too: τ^2-bench measured simulator error rates of 40% in retail and 47% in airline tasks.

Yao et al., "τ-bench," arXiv:2406.12045. Barres et al., "τ^2-bench," arXiv:2506.07982. Sierra Research, τ^2-bench leaderboard (Aug 2026). Wu et al., "MCPMark," arXiv:2509.24002.

Execution Feedback and Recovery: When Tools Fail

Error source	Types (CRITICTOOL)
Internal (the model)	tool selection, tool hallucination, parameter key, parameter value
External (the world)	environment errors such as timeouts and permission failures

Best overall score: GPT-4o at 69.01. External errors are handled worst.

- Recovery falls about **37%** when the failure is *implicit* and semantic, not an explicit error.
- Fault tolerance improves **3.66× more slowly** than nominal task success.
- When the API itself drifts, GPT-5.4 loses 13.7% and Claude Sonnet 4.6 loses 14.4%.

- **The reasoning trap.** Strengthening reasoning with RL raises tool hallucination in proportion to the task gains.
- **Knowing when to stop.** With insufficient information, GPT-4o scores 42.0 in ToolSandbox: it invents arguments.

Huang et al., "CRITICTOOL," arXiv:2506.13977. Zhu et al., "ToolMaze," arXiv:2606.05806. Yin et al., "The Reasoning Trap," arXiv:2510.22977. Liu et al., "MCPEvol-Bench," arXiv:2607.14642. Lu et al., "ToolSandbox," arXiv:2408.04682.

Six Axes, and the Benchmark for Each

Axis	How it is measured	A number to remember
Schema adherence	BFCL syntax-tree match, JSONSchemaBench	hard-schema coverage 0.09 to 0.41
Tool selection	hallucination and irrelevance tests	2023 models invented APIs 36 to 88% of the time
Argument correctness	ComplexFuncBench, NESTFUL	nested sequences: best full match 28%
Execution feedback	backend state checks (BFCL v3, τ-bench)	state, not the transcript, is graded
Recovery	error injection (BFCL v4), ToolMaze	recovery down 37% on implicit failures
Reliability	pass^k over repeated trials	the gap to one trial has not closed

- **The trend:** failures move from wrong arguments toward reasoning. In MCP-Atlas, 63.3% of failures are cognitive.

Berkeley Gorilla team, BFCL blogs. Patil et al., arXiv:2305.15334. Basu et al., "NESTFUL," arXiv:2409.03797. Zhong et al., "ComplexFuncBench," arXiv:2501.10132. Bandi et al., "MCP-Atlas," arXiv:2602.00933.

Tools: What to Remember

- Tool learning went from a **loss filter** to **executed synthetic data** to **RL rewards**. Execution-based verification is the constant.
- Tool *discovery* is now a **retrieval problem**: defer schemas, search for tools, call them from code.
- Constrained decoding solved **syntax**. Selection, argument values and recovery remain open.
- Measure **pass^k**, not one lucky run, and grade the **state** the agent leaves behind.
- Stronger reasoning does not imply safer tool use. RL can amplify tool hallucination.

Next. Tools now arrive through one shared protocol. That solves integration, and it creates a new trust boundary.

Agentic Models: **Interoperability and MCP**

One protocol now connects any agent to any tool. Who is trusted across that connection, and for what?

What Changed in the 2026-07-28 Specification

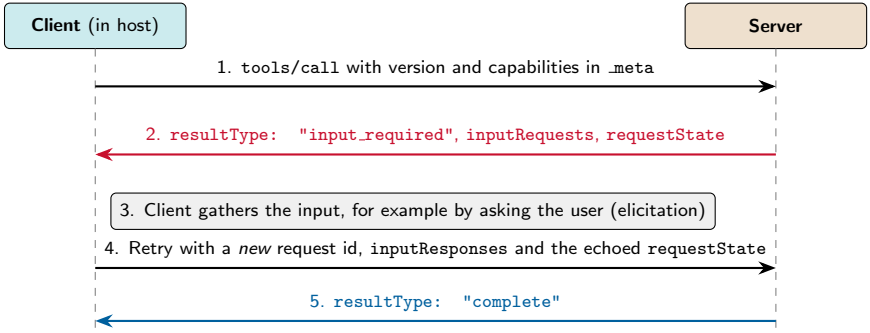
- **Governance.** Anthropic donated MCP to the Agentic AI Foundation (Linux Foundation) in December 2025, at 10,000+ active public servers and 97M monthly SDK downloads.
- **Unchanged:** host, client and server roles; tools, resources and prompts.

Changed	Before	Now
Lifecycle	`initialize` handshake	removed. Version and capabilities in every request
State	protocol sessions	removed. Server-minted handles in tool arguments
Client features	Sampling, Roots, Logging	deprecated. Elicitation stays
Transport	HTTP+SSE stream	deprecated. Each message is one POST
Registration	Dynamic Client Registration	deprecated. Client ID Metadata Documents first

The theme is statelessness. Deprecated features stay for at least twelve months before removal.

Anthropic, "Donating the Model Context Protocol and establishing the Agentic AI Foundation" (Dec 2025). MCP, "Key Changes" changelog, specification 2026-07-28. Soria Parra and Delimarsky, "The 2026-07-28 Specification" (MCP blog).

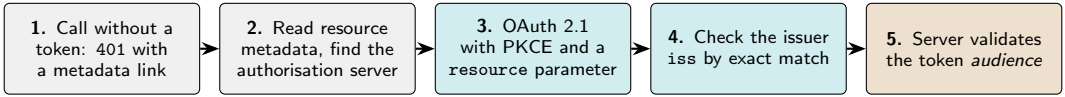
Stateless Does Not Mean One Shot: Multi Round-Trip Requests



- **Why.** With no session, the server cannot remember a half-finished call. The state travels through the client instead.
- **The new attack surface.** The server must treat `requestState` as attacker-controlled, and protect its integrity.

MCP Specification 2026-07-28, "Multi Round-Trip Requests" and "Key Changes."

Authorisation: Tokens Bound to One Server



- **Roles.** The MCP server is an OAuth 2.1 *resource server*. The MCP client is an OAuth client, inside a host application that owns the user's permissions and consent.
- **Audience binding.** A server must not accept, or pass on, a token issued for anything else. This forbids token passthrough.
- **Step-up.** Start with minimal scope, and re-authorise only when the server asks for more.

The protocol states its own limit: MCP *"cannot enforce these security principles at the protocol level."* The host must build consent.

MCP Specification 2026-07-28, "Authorization." MCP, "Security Best Practices."

Trust Boundaries: The Official Threat Catalogue

Threat	How it works	Mitigation
Confused deputy	a proxy's consent cookie skips the consent screen. The code goes to the attacker	per-client consent, exact redirect matching
Token passthrough	server forwards the client's token downstream	forbidden. It breaks audience checks and audit trails
Server-side request forgery	malicious server puts internal URLs in its metadata	HTTPS only, block private ranges, egress proxy
State-handle hijacking	holding a handle is treated as authentication	a handle is "a name, not a capability"
Local server compromise	one-click install runs an arbitrary command	show the full command, sandbox, least privilege

Every row is a place where **who is acting** and **on whose authority** come apart.

MCP, "Security Best Practices" (current page). MCP Specification 2026-07-28, "Tools."

Attacks That Were Demonstrated

- **Tool poisoning.** Instructions hidden in a tool *description*: invisible to the user, visible to the model.
- **Rug pull.** The description changes *after* the user approved the tool.
- **Shadowing.** One server's description changes how a *trusted* server's tool is used.
- **GitHub MCP.** A malicious public issue led an agent to leak private repositories.
- **Supabase MCP.** A support ticket made an over-privileged agent post secret tokens back into the ticket.

Measured in 2025 to 2026	
Highest attack success, MCPTox	72.8%
Highest refusal rate, MCPTox	under 3%
Remote servers with no authentication	40.55%
Servers flagged risky by scanners	96.89%
Scanner alerts that are real	under 50%

MCPTox: *"more capable models are often more susceptible."* Architecture fixes this, not capability.

Invariant Labs, tool-poisoning and GitHub MCP reports (2025). General Analysis, Supabase MCP report (2025). Wang et al., "MCPTox," arXiv:2508.14925. Zhou et al., arXiv:2605.22333. Chen et al., arXiv:2607.11086.

Permissions at the Boundary

- **Asking the user, safely.** Elicitation *forms* must not request passwords or keys. Those go through a *URL* flow that neither the client nor the model can inspect.
- **Deterministic rules in the host.** Claude Code rules name a server or a tool, such as `mcp__puppeteer`. An unanchored wildcard allow rule is ignored.
- **Supply side.** The MCP Registry is still a preview, and it delegates security scanning to package registries.
- **Pin what you approved.** Show users what the model sees, and pin tools by hash.

Scanners alone are weak assurance. Pair them with **pinning** and **least privilege**.

MCP Specification 2026-07-28, "Elicitation." Anthropic, Claude Code permissions docs. MCP Registry, "About" (Oct 2026). Invariant Labs (Apr 2025).

MCP: What to Remember

- The 2026-07-28 specification made MCP **stateless**: no handshake, no sessions, state in handles and in `requestState`.
- Authorisation is OAuth 2.1 with **audience-bound tokens**. Token passthrough is forbidden.
- The protocol cannot enforce consent. **The host is the trust boundary.**
- Tool descriptions are **untrusted input** that the model reads and the user does not see.
- Stronger models did not resist poisoning better. Least privilege, pinning and data-flow control do the work.

Next. Tools give the agent reach. Memory gives it a past. What should an agent keep once the context window ends?

Agentic Models: **Memory Systems**

The context window forgets when the session ends. What should an agent keep, where, and who decides?

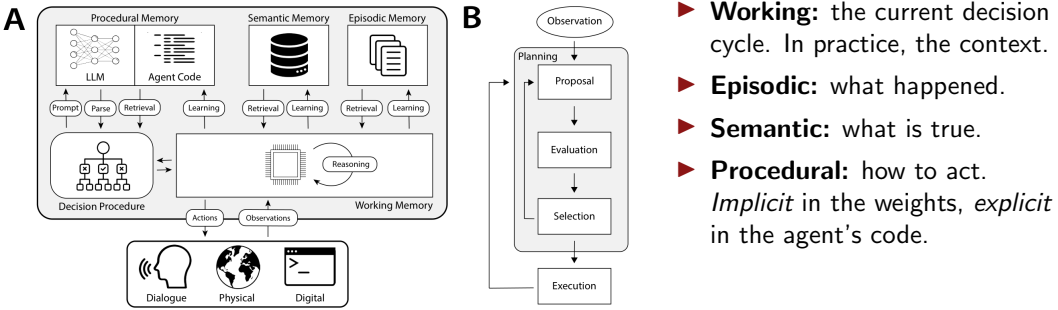
Memory Is Not the Same as a Long Context

- **The tempting answer:** context windows reach a million tokens, so put everything in.
- **Why it fails.**
  - *LongMemEval.* Assistants and long-context models drop **30 to 60%** on 115K-token chat histories, against seeing only the relevant evidence.
  - *Cost.* Full context on LoCoMo: 17 s at p95, against 1.4 s for a memory layer.
- **What memory must do** (MemoryAgentBench): accurate retrieval, test-time learning, long-range understanding, and selective forgetting.

The design question for memory: what should the agent **write down**, so that it does not have to read everything again?

Wu et al., "LongMemEval," arXiv:2410.10813. Chhikara et al., "Mem0," arXiv:2504.19413. Hu et al., "MemoryAgentBench," arXiv:2507.05257.

A Vocabulary: Episodic, Semantic and Procedural Memory



- **Working:** the current decision cycle. In practice, the context.
- **Episodic:** what happened.
- **Semantic:** what is true.
- **Procedural:** how to act. *Implicit* in the weights, *explicit* in the agent's code.

- Memory is reached by internal actions: *retrieval* reads it, *learning* writes it. Writing procedural memory is the riskiest: it can introduce bugs or subvert the designer's intent.

Sumers, Yao, Narasimhan and Griffiths, "Cognitive Architectures for Language Agents" (CoALA), arXiv:2309.02427, Figure 4.

Where Each Memory Type Lives in Real Systems

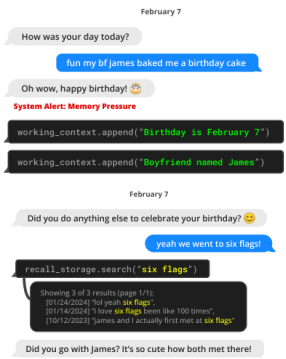
Type	Research systems	Products
Working	MemGPT working context	the context window
Episodic	MemGPT recall storage, Generative Agents memory stream	Claude chat search, ChatGPT chat history
Semantic	Mem0 facts, Zep entity graph	Claude topics, ChatGPT saved memories
Procedural, explicit	Voyager skills, Agent Workflow Memory	Agent Skills
Procedural, implicit	fine-tuning and agentic RL	the model weights

- **Episodic memory is the neglected one.** A 2025 position paper calls it the missing piece for long-term agents.

Mapping is a synthesis of the cited systems, after Sumers et al., arXiv:2309.02427. Pink et al., "Position: Episodic Memory is the Missing Piece," arXiv:2502.06975.

Lineage 1: The Agent Manages Its Own Memory (MemGPT)

- **Idea.** Borrow the operating-system memory hierarchy. The LLM pages information in and out with *function calls*.
- **Main context** (the prompt): read-only instructions, a fixed-size *working context* the model can edit, and a FIFO queue of messages.
- **External context:** *recall storage* (every message) and *archival storage* (arbitrary text).
- **Memory pressure.** A warning fires at about 70% of the window. At 100% the queue is flushed and summarised.
- **Result.** Deep memory retrieval with GPT-4: 32.1% without, **92.5%** with.



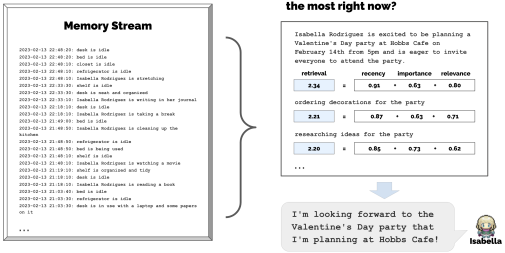
Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, Figures 1 and 3, Table 2. Top: a write after a pressure alert. Bottom: a search of recall storage.

Lineage 2: A Pipeline Manages Memory (Generative Agents)

- **Memory stream.** A time-stamped log of observations, plans and reflections.
- **Retrieval score**, each term normalised to $[0, 1]$:

$$\text{score} = \text{recency} + \text{importance} + \text{relevance}$$

- Recency decays by 0.995 per game hour. Importance is rated 1 to 10 by the LLM.
- **Reflection.** When summed importance passes 150, the agent writes 5 insights. Episodes become *semantic* memory.



An origin idea. Systems since 2025 replace the hand-tuned score with **LLM-decided update operations** and temporal invalidation.

Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," arXiv:2304.03442, Figure 6.

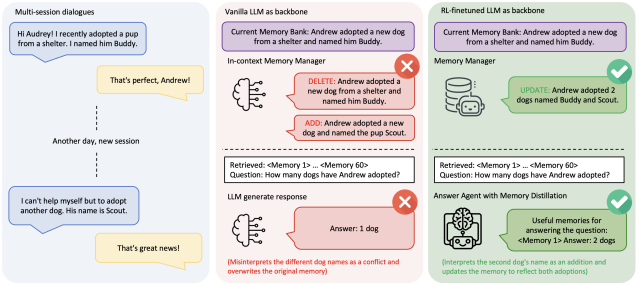
The Write Path: Extract, Compare, Mutate

System	Store	What happens on a write
Mem0	facts, optionally a graph	LLM extracts facts, then chooses `ADD`, `UPDATE`, `DELETE` or `NOOP`
A-MEM	linked notes	the LLM links a new note to similar ones, then *evolves* the old notes
Zep	temporal knowledge graph	on a contradiction, the old edge is *invalidated*, not deleted

- **What changed.** "Append and embed" became an LLM-in-the-loop write. Cost moves from read time to write time.
- **Asynchronous consolidation.** Letta's sleep-time agent rewrites memory between user turns: about 5× less test-time compute for the same accuracy.

Chhikara et al., arXiv:2504.19413. Xu et al., "A-Mem," arXiv:2502.12110. Rasmussen et al., "Zep," arXiv:2501.13956. Lin et al., "Sleep-time Compute," arXiv:2504.13171.

Learning the Write Policy with Reinforcement Learning

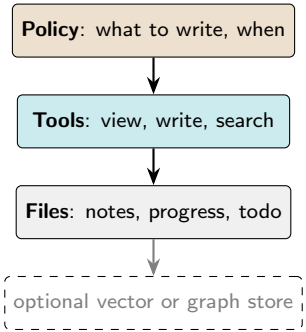


Method	What the policy controls	Headline, from a final-answer reward only
Memory-R1 (figure)	discrete operations on a store	+48% F1 over Mem0, 152 training pairs
MEM1	a state that *replaces* the context	3.5× performance, 3.7× less memory
MemAgent	a 1,024-token overwrite buffer	trained at 8K, 71% at 3.5M tokens

Figure: Yan et al., "Memory-R1," arXiv:2508.19828, Figure 1. Zhou et al., "MEM1," arXiv:2506.15841. Yu et al., "MemAgent," arXiv:2507.02259. Some numbers were extracted by a summarising tool.

Industry Converged on Something Simpler: Files, Tools, a Policy

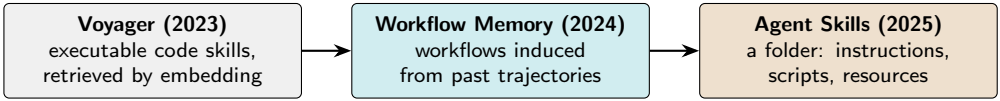
- **The file system as memory.** Manus recites a `todo.md` across about 50 tool calls per task. Compression is *restorable*: keep the URL, drop the page.
- **A memory tool.** Claude's API exposes a `/memories` directory. The injected protocol: view memory first, assume interruption.
- **The surprise.** An agent with plain `grep` and file tools scored 74.0% on LoCoMo, above a dedicated graph memory at 68.5%.



A 2026 study of 12 systems on 5 workloads: **no single architecture dominates**.

Ji, "Context Engineering for AI Agents: Lessons from Building Manus" (2025). Anthropic, memory tool docs. Letta, "Benchmarking AI Agent Memory" (2025, vendor). Zhou et al., "Are We Ready For An Agent-Native Memory System?" arXiv:2606.24775.

Procedural Memory: Skills the Agent Can Reuse



- **Voyager.** A growing skill library gave 3.3× more unique items and 15.3× faster tech-tree milestones in Minecraft.
- **Agent Workflow Memory.** Relative success up 24.6% on Mind2Web and 51.1% on WebArena.
- **Agent Skills** use *progressive disclosure*: only a name and description sit in the prompt. Instructions and scripts load on demand, like deferred tools.
- A skill from an untrusted source can carry malicious instructions.

Wang et al., "Voyager," arXiv:2305.16291. Wang, Mao, Fried and Neubig, "Agent Workflow Memory," arXiv:2409.07429. Anthropic, "Equipping agents for the real world with Agent Skills" (Oct 2025).

Retrieval-Memory Hybrids: Give the Index a Structure

System	Structure built at index time	Memory role	Headline
RAPTOR	tree of recursive cluster summaries	consolidation	+20% absolute on QuALITY
MemoRAG	a small model's compressed global memory drafts retrieval clues	global memory	40.2 against 35.0 for full context

- Graph indexes play the same two roles: GraphRAG's community summaries are **consolidation**, HippoRAG's knowledge graph is **association**.

Flat RAG only recognises chunks. Each of these **precomputes a kind of memory** for sense-making or association.

Sarthi et al., arXiv:2401.18059. Edge et al., arXiv:2404.16130. Jiménez Gutiérrez et al., arXiv:2405.14831. Qian et al., arXiv:2409.05591. Su, Berkeley CS294 (2025).

Structure Has a Price

- **Simple facts get worse.** GraphRAG, LightRAG and RAPTOR drop on plain factual questions against a strong dense retriever.
- **Graph indexes are expensive.** A GraphRAG answer reads about 610,000 tokens of community reports. LightRAG needs under 100.

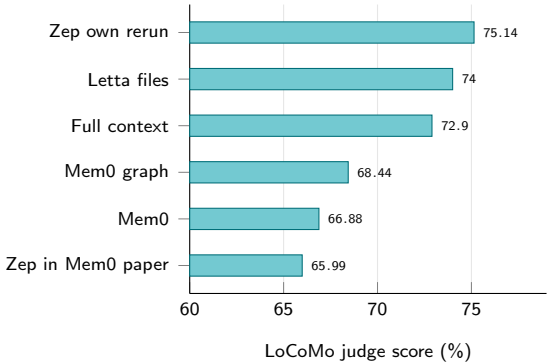
MemoryAgentBench	Retrieval	Overall
BM25	60.5	
GraphRAG	40.9	19.9
MIRIX	47.5	34.3

MIRIX self-reports 85.4% on LoCoMo.

Not all memory should be external. On deep compositions of facts, models using retrieval "fail badly". A grokked transformer is near perfect.

Jiménez Gutiérrez et al., "HippoRAG 2," arXiv:2502.14802. Guo et al., "LightRAG," arXiv:2410.05779. Hu et al., arXiv:2507.05257. Su, Berkeley CS294 (2025).

The LoCoMo Dispute: Read Memory Benchmarks Carefully



- In Mem0's own paper, full context beats every memory layer.
- LoCoMo conversations are only 16K to 26K tokens. They fit in context, and they contain no knowledge-update tests.
- **Defensible claim:** memory layers trade about 4 to 7 points for roughly 10× lower latency and far fewer tokens.

Every number here is **vendor-reported**. There is no independent replication. Prefer LongMemEval and MemoryAgentBench.

Chhikara et al., arXiv:2504.19413, Table 2. Chalef and Rasmussen, "Is Mem0 Really SOTA in Agent Memory?" (Zep blog, 2025). Letta, "Benchmarking AI Agent Memory" (2025). Maharana et al., "LoCoMo," arXiv:2402.17753.

Memory in Products, and What Nobody Has Solved

Product	Two mechanisms	User controls
Claude	topics saved during chats, plus chat search as a tool	editable topics, incognito chats, sensitive categories excluded by default
ChatGPT	saved memories, plus reference to chat history	temporary chats, memory sources shown, auto-updating since June 2026
Gemini	personal context from past chats	on by default, Temporary Chats

- **Forgetting is the weakest skill.** Multi-hop conflict resolution peaks at about 28% for every method in MemoryAgentBench.
- **Governance gap.** A 2026 survey of 435 works finds research focused on accumulating state, not on governing, recovering or relinquishing it.

Anthropic Help Center, chat search and memory (2026). OpenAI, Memory FAQ and 2026 release notes. Google, Gemini Temporary Chats (2025). Hu et al., arXiv:2507.05257. Ding et al., "Always-On Agents," arXiv:2606.30306.

Memory: What to Remember

- A long context is not a memory. Models lose 30 to 60% on long histories, and context has a cost.
- **Three types** to design for: episodic (what happened), semantic (what is true), procedural (how to act).
- Systems differ most in their **write policy**: hand-tuned scores, LLM-chosen operations, or a policy learned with RL.
- Structured retrieval adds sense-making and association, and pays in indexing cost and simple-fact accuracy.
- In practice, memory is **files, tools and a policy**. Benchmarks in this area are vendor-run and disputed.

Next. A sub-agent with its own context window is also a memory strategy. When is splitting the work across agents worth it?

Agentic Models: **Multi-Agent Collaboration**

When does a second agent help, and what does it cost?

Why Use More Than One Agent?

- **The case for.** Anthropic's research system, an Opus 4 lead with Sonnet 4 sub-agents, scored **90.2% better** than a single Opus 4 agent on an internal research evaluation.
- **The bill.** About 4× the tokens of chat for one agent, about 15× for many.
- On BrowseComp, token usage explains **80%** of the variance.
- **Two motives** (Noam Brown): *latency*, since chains of thought are serial but agents run in parallel, and *diversity*. "Routing is already a form of multi-agent AI."

Architecture	How agents interact
Single agent	one loop
Independent	parallel, merged only at the end
Centralised	an orchestrator directs and verifies
Decentralised	peers debate to consensus
Hybrid	hierarchy plus limited peer talk

The question for this section: is the gain from **collaboration**, or from **more tokens in parallel**?

Hadfield et al., "How we built our multi-agent research system" (Anthropic, Jun 2025). Brown, "Multi-Agent AI," Berkeley Agentic AI (Oct 2025). Kim et al., "Towards a Science of Scaling Agent Systems," arXiv:2512.08296.

Collaboration Patterns I: Debate and Aggregation

Task	Single	Reflect	Debate
Arithmetic	67.0	72.1	**81.8**
GSM8K	77.0	75.0	**85.0**
MMLU	63.9	57.7	**71.1**

Multi-agent debate, 3 agents and 2 rounds (2023).

- **2023 claim.** Agents that critique each other converge on better answers.
- **2025 reality check.** Split debate into *voting* plus *talking*. Across 7 benchmarks, majority voting alone explains most of the gain.

- **Same story for aggregation.** Mixture-of-Agents layered six open models to a 65.1% win rate on AlpacaEval 2.0, above GPT-4o at 57.5%.
- **Self-MoA** then aggregated samples from the *single best* model, and beat it by 6.6%.

Much of the benefit is **parallel test-time compute**, the gain that self-consistency and best-of-N get from a single model.

Du et al., arXiv:2305.14325. Choi, Zhu and Li, "Debate or Vote," arXiv:2508.17536. Wang et al., "Mixture-of-Agents," arXiv:2406.04692. Li et al., "Rethinking Mixture-of-Agents," arXiv:2502.00674.

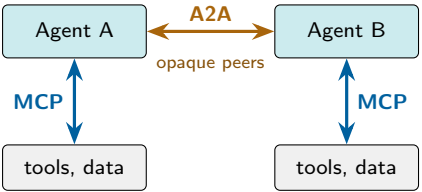
Collaboration Patterns II: From Role-Play to Three Primitives

2023 system	Mechanism	Survives in 2026 as
MetaGPT	roles follow a procedure and exchange structured documents	workflow orchestration
ChatDev	a fixed chain of design, coding and testing chats	workflow orchestration
AutoGen	agents auto-reply. A manager picks the next speaker	group chat (Microsoft Agent Framework 1.0)

- **Three primitives remain:** a *handoff* transfers control, an *agent-as-tool* is called and returns, a *workflow* schedules many agents.
- **Where the gains came from:** a **verifier or grounding agent**, and **role specialisation**. ChatDev's quality falls from 0.40 to 0.22 without roles.

Li et al., "CAMEL," arXiv:2303.17760. Hong et al., "MetaGPT," arXiv:2308.00352. Qian et al., "ChatDev," arXiv:2307.07924. Wu et al., "AutoGen," arXiv:2308.08155. OpenAI Agents SDK, handoffs docs. Microsoft Agent Framework 1.0 (Apr 2026).

Communication Protocols: MCP Is Vertical, A2A Is Horizontal



Apr 2025	A2A launched by Google
Jun 2025	moves to the Linux Foundation
Aug 2025	IBM's ACP merges into A2A
Apr 2026	v1.0, with signed Agent Cards
Aug 2026	joins the Agentic AI Foundation

- **A2A objects.**
  - *Agent Card*: identity, skills, endpoint, authentication
  - *Task*: a unit of work with states such as working, input required, completed, failed
  - *Message* and *Artifact*: built from typed parts
- **Opaque execution.** Agents do not share thoughts, plans or tool implementations.

A2A keeps agents opaque, but Cognition argues that agents should share *full traces*.

A2A Project, specification v1.0.0 and "A2A and MCP." Linux Foundation press releases (Jun 2025, Apr 2026). IBM BeeAI, "ACP Joins Forces with A2A" (Aug 2025). Yan, "Don't Build Multi-Agents" (Cognition, 2025).

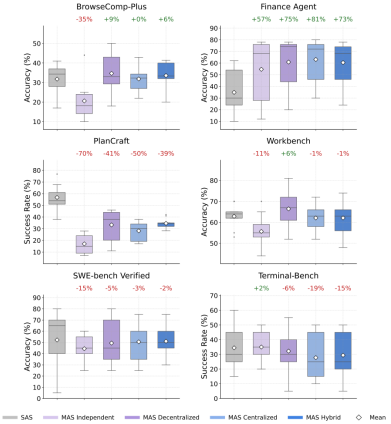
Coordination Costs, Measured

Metric	Single	Independent	Decentralised	Centralised	Hybrid
Token overhead	0%	58%	263%	285%	515%
Coordination efficiency	**0.466**	0.234	0.132	0.120	0.074
Successes per 1K tokens	**67.7**	42.4	23.9	21.5	13.6
Error amplification	1.0	17.2	7.8	4.4	5.1

- **Capability saturation.** Once a single agent scores above about **45%**, adding agents gives negative returns.
- **Tools against coordination.** The more tool-heavy the task, the more overhead hurts.
- **Errors amplify by topology.** Independent agents propagate errors most. A central verifier contains them.

Kim et al., "Towards a Science of Scaling Agent Systems," arXiv:2512.08296 (v3, Apr 2026, which supersedes the earlier blog numbers), Table 5. 260 configurations on 6 benchmarks.

It Depends on the Task



- **Parallel, read-heavy work wins.** Finance-Agent gains 80.8% with a centralised team.
- **Sequential, stateful work loses.** Every multi-agent variant degrades PlanCraft, by 39 to 70%.
- **Coding is neutral or worse.** SWE-bench Verified: −2.1% at best.
- **Read against write:** research parallelises. Code does not, because actions carry *implicit decisions* that conflict.

Figure: Kim et al., arXiv:2512.08296, Figure 2. Yan, "Don't Build Multi-Agents" (Cognition, Jun 2025). Chase, "How and when to build multi-agent systems" (LangChain, Jun 2025).

Failure Modes: The MAST Taxonomy

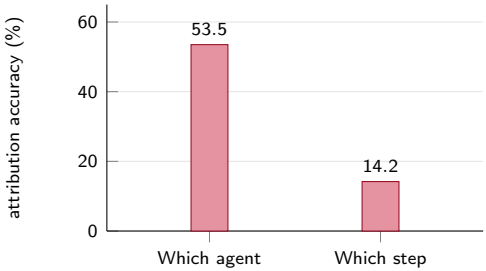
Category	Most frequent modes	Share
System design	step repetition	15.7%
	unaware of termination conditions	12.4%
Inter-agent misalignment	reasoning and action mismatch	13.2%
	task derailment	7.4%
Task verification	incorrect verification	9.1%
	no or incomplete verification	8.2%

- Built from 150+ traces with six experts. Agreement $\kappa = 0.88$.
- **Failure rates** across seven systems: from 41% (MetaGPT) to 86.7% (HyperAgent).

These are **specification, hand-over and verification** failures: organisational problems, on simple tasks.

Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" arXiv:2503.13657 (v3). 14 modes in total. The two most frequent per category are shown.

Finding the Culprit, and Fixing It



Best automated failure attribution on logs from 127 multi-agent systems (Who&When).

- **Debugging is unsolved.** The best method finds the responsible agent half the time, and the decisive step rarely.
- **Tactical fixes help modestly.** A final consensus step: +9.4%. High-level objective verification: +15.6%.
- **Structural fixes help more.** Explicit task boundaries for sub-agents, one agent for final synthesis, a central verifier.

A fair evaluation compares at matched token budgets against one agent with self-consistency.

Zhang et al., "Which Agent Causes Task Failures and When?" arXiv:2505.00212. Cemri et al., arXiv:2503.13657, Table 5. Hadfield et al. (Anthropic, 2025).

Multi-Agent Systems: What to Remember

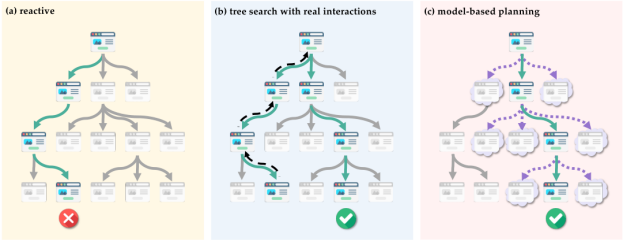
- Multi-agent systems **buy accuracy with tokens**: about 15× chat, with 58 to 515% coordination overhead.
- Debate and model mixing mostly reduce to **voting and sampling**. Specialised roles and verifiers are what add value.
- They pay off on **breadth-first, read-only, parallel** tasks where one agent is weak. They hurt sequential and write-heavy work.
- **A2A** connects opaque agents horizontally. **MCP** connects an agent to tools vertically. Both now sit in one foundation.
- Failures are organisational, and attributing them automatically does not yet work.

Next. So far the agent acts and then sees what happened. Can it predict the outcome first, and can we train it in a simulated world?

Agentic Models: **World Models and Environments**

Can an agent try an action in its head before it tries it in the world?

Three Ways to Choose the Next Action



- For LLM agents, a **world model** predicts state changes *in text*.
- Reactive agents are greedy. Real tree search is systematic, but real actions can be irreversible, unsafe and slow. So simulate first.

Figure: Gu et al., "Is Your LLM Secretly a World Model of the Internet?" (WebDreamer), arXiv:2411.06559, Figure 1. Su, "On Memory, Reasoning, and Planning of Language Agents," Berkeley CS294 (Feb 2025).

One-Step Lookahead on the Web

- **WebDreamer.** For each candidate action, the LLM imagines the outcome, scores it (1.0 complete, 0.5 on track, 0 wrong), and takes the best:

$$a_t = \arg\max_a \ \mathrm{score}\big(\mathrm{sim}(o_t, a)\big)$$

- **The best horizon is one step.** At two or three steps, errors accumulate.
- **WMA** predicts only *what changes* between pages, in natural language. It costs \$0.4 per task against \$2.7 for tree search.

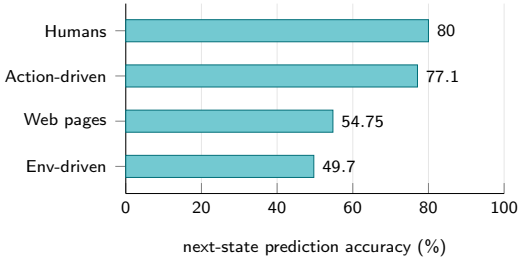
Success (%)	Reactive	Tree	Dreamer
VisualWebArena	17.6	**26.2**	23.6
Online-Mind2Web	26.0	n/a	**37.0**
Mind2Web-Live	20.2	n/a	**25.0**
Seconds per task	79.8	835.7	198.9

Tree search cannot run on live sites: actions there cannot be undone.

Simulation recovers most of the benefit of search, and it is the **only** option when actions are irreversible.

Gu et al., arXiv:2411.06559, Tables 1 and 2 (time is a mean over three sites). Chae et al., "Web Agents with World Models," arXiv:2410.13232.

How Good Are LLM Simulators?



GPT-4 on text-game transitions. Web pages: average over LLMs.

- LLMs predict what *their own action* does. They get only about half right on what the *environment* does next by itself.
- **Do they hold a coherent world model?** A transformer trained on New York taxi routes predicts next turns perfectly, yet its implied street map is incoherent.
- With 10% random detours, its valid-route rate falls to 0.08.

Today's LLM world models are *local, one-step critics*.

Wang et al., "Can Language Models Serve as Text-Based World Simulators?" arXiv:2406.06485. Chae et al., arXiv:2410.13232. Vafa et al., "Evaluating the World Model Implicit in a Generative Model," arXiv:2406.03689.

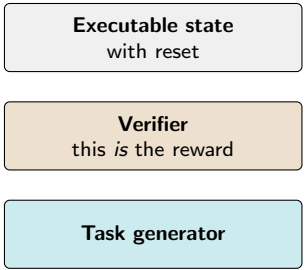
Environment Interaction: Computer Use as the Hard Case

Benchmark	Setup	Result
OSWorld (Apr 2024)	369 tasks in real virtual machines	humans 72.36%, best model 12.24%
OSWorld-Verified (Jul 2025)	300+ issues fixed	best agent 60.76%
OSWorld 2.0 (Jun 2026)	108 tasks, median 1.6 human hours	Claude Opus 4.8: **20.6%** complete, 54.8% partial

- **What failed in 2024:** over 75% of analysed failures were inaccurate clicks. A *grounding* problem.
- **What fails in 2026:** losing track of constraints. All models score zero on tasks beyond 163 minutes.

Xie et al., "OSWorld," arXiv:2404.07972. XLANG Lab, "Introducing OSWorld-Verified" (Jul 2025). Yuan et al., "OSWorld 2.0," arXiv:2606.29537.

Environments Became Training Infrastructure



three parts of an environment

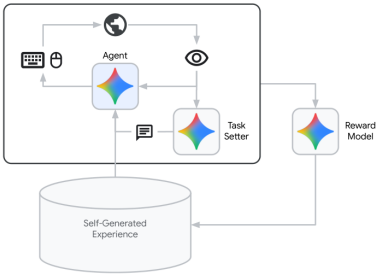
- **SWE-Gym, R2E-Gym.** Thousands of repository tasks with runnable tests. R2E-Gym generates 8.7K+ of them procedurally from commits.
- **Open environments.** Prime Intellect's hub packages each one as dataset, rubric and interaction loop: 400+ in two months.
- **World modelling as a training objective.** Meta's Code World Model spends about 5T tokens on predicting Python execution traces line by line.

This is the supply side of the open-lab agentic RL recipe: **the environment is the dataset**.

Pan et al., "SWE-Gym," arXiv:2412.21139. Jain et al., "R2E-Gym," arXiv:2504.07164. Prime Intellect, Environments Hub posts (2025). Meta FAIR CodeGen, "CWM," arXiv:2510.02387, and model card.

Training in Imagination

- **DreamGym.** An LLM *experience model* generates states and rewards in abstract text. With zero real interactions, WebShop reaches 63.9, against 65.0 for RL on 80K real transitions (Llama-3.1-8B).
- **DynaWeb.** Mix imagined rollouts with real ones. About 40% real data is the sweet spot.
- **SIMA 2** (right). A Gemini task setter proposes tasks, a Gemini reward model scores them, and the agent retrains on its successes, including inside Genie 3 generated worlds.



Simulators are **sycophantic**: WebWorld reports "overly optimistic outcomes that cater to the agent's action".

Chen et al., "DreamGym," arXiv:2511.03773. Ding et al., "DynaWeb," arXiv:2601.22149. Xiao et al., "WebWorld," arXiv:2602.14721. Figure: Google DeepMind SIMA team, "SIMA 2," arXiv:2512.04797, Figure 16.

World Models: What to Remember

Situation	Reasonable choice (our synthesis)
Environment is cheap to reset, policy is strong	stay reactive and model-free
Actions are irreversible or slow	one-step lookahead with an LLM world model
No RL infrastructure, or real data is scarce	synthetic experience, always mixed with real data

- LLM world models work as **local critics**. They are weak on environment-driven change and on long rollouts.
- The same trained model now serves as inference-time critic, synthetic-data generator and environment substitute.

Next. Every claim so far rested on a benchmark number. How far can those be trusted?

Agentic Models: **Evaluation and Benchmarking**

A leaderboard says 80%. Eighty percent of what, measured how, and how many times?

The Benchmark Lifecycle: Saturate, Audit, Replace

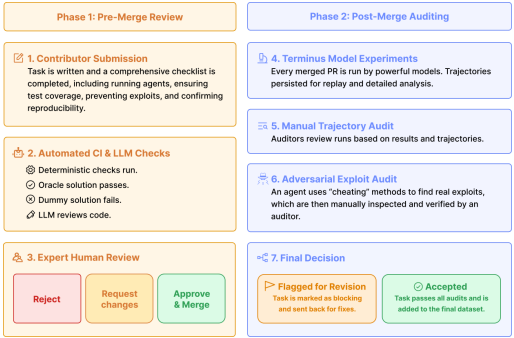
Family	First wave	Now	Successor
SWE-bench Verified	SWE-agent 12.5% on the original (2024)	above 80% (2026)	SWE-Bench Pro: best 61.5% $\pm$3.1
GAIA	GPT-4 with plugins 15%, humans 92%	76.4% (2025). The board looks inactive	
BrowseComp	GPT-4o with browsing 1.9%	86.8% with a multi-agent harness	

- OSWorld and τ-bench followed the same path, and were succeeded by OSWorld 2.0 and τ^2-bench.
- A benchmark's useful life is about **12 to 18 months** (our estimate). These benchmarks measure *acting* in an environment.

Deng et al., "SWE-Bench Pro," arXiv:2509.16941, and leaderboard (Oct 2026). Mialon et al., "GAIA," arXiv:2311.12983. Wei et al., "BrowseComp," arXiv:2504.12516. Anthropic, Opus 4.6 announcement.

Anatomy of a Modern Agent Task

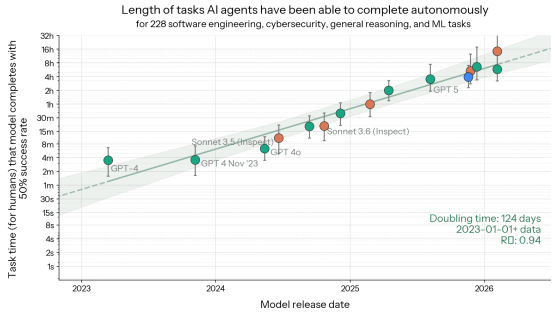
- **A Terminal-Bench 2.0 task** is an instruction, a Docker image, tests and an oracle solution. Tests look only at the *final container state*.
- **Three properties to verify:**
  - *Specificity*: tests pass if and only if the outcome is acceptable
  - *Solvability*: the oracle passes
  - *Integrity*: no shortcuts exist
- About **3 reviewer-hours per task**. 89 tasks kept out of 229 contributed.



Building a trustworthy agent benchmark is now **security engineering**: assume the agent will find any shortcut you left.

Merrill et al., "Terminal-Bench," arXiv:2601.11868, Figures 2 and 3.

Time Horizons: Measuring Autonomy in Human Hours



Horizon (min)	50%	80%
Claude 3.7 Sonnet	60	12
GPT-5	203	38
Claude Opus 4.5	293	49
Gemini 3.1 Pro	384	90
Claude Opus 4.6	719	70
Mythos Preview	1,045	186

- **Doubling time:** 7 months fitted since 2019, 128.7 days since 2023. Above about 16 h the suite is unreliable.

- **Method.** Time human experts per task, fit model success against log task length, read off the 50% (or 80%) crossing.

- Read the intervals. Opus 4.6's 50% horizon: confidence interval 317 to 3,634 minutes.

Kwa et al., "Measuring AI Ability to Complete Long Software Tasks," arXiv:2503.14499. Figure: METR, live chart (its fitted doubling time differs slightly from the table's source). Table, doubling times and reliable range: METR, Time Horizon 1.1 data file (suite updated 8 May 2026).

The Harness Is Part of the Score

Model	Terminal-Bench 2.0 score by harness	Spread
GPT-5	Codex CLI 49.6, OpenHands 41.5, Terminus 2 35.2, Mini-SWE-Agent 33.9	15.7 points
Claude Opus 4.5	Terminus 2 57.8, Claude Code 52.1, OpenHands 51.9	5.9 points
Gemini 2.5 Pro	Terminus 2 32.6, Mini-SWE-Agent 26.1, Gemini CLI 19.6, OpenHands 15.7	2.1×

- **Tokens vary even more.** Opus 4.5 used 3.9M input tokens on one harness and 256.9M on another.
- **Leaderboard ranks are often ties.** SWE-Bench Pro intervals are about ±3.6 points, and five models share rank 5.

A score is a tuple: model, scaffold, budget, trials, task subset, grader version, confidence interval.

Merrill et al., arXiv:2601.11868, Table 2. Scale AI, SWE-Bench Pro public leaderboard (Oct 2026).

Cost and Reliability Are First-Class Axes

Cost

- HAL's summary: agents "can be 100x more expensive while only being 1% better".
- In **21 of 36** model, agent and benchmark combinations, more reasoning effort gave equal or *lower* accuracy.

Reliability

- Over 24 months, accuracy climbed while reliability improved only slightly.
- Agents tolerate infrastructure faults but break under paraphrase of the task.

Dimension	Reliability metrics (12 in total)
Consistency	same outcome, same trajectory, same resource use across runs
Robustness	fault injection, environment perturbation, paraphrase
Predictability	calibration, discrimination, Brier score
Safety	compliance, harm severity

Kapoor et al., "Holistic Agent Leaderboard," arXiv:2510.11977. Rabanser et al., "Towards a Science of AI Agent Reliability," arXiv:2602.16666.

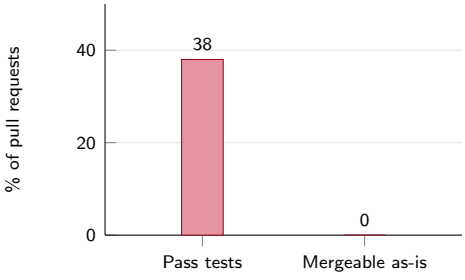
Broken Benchmarks, Quantified

Benchmark	Flaw	Effect
τ-bench	an empty response counts as success on impossible tasks	38% overestimate
SWE-bench Verified	insufficient tests	24% of top-50 ranks change
SWE-Lancer	the agent can read the test files	100% without solving anything
OSWorld	selectors broke when websites changed	28% *under*estimate

- Of 10 benchmarks audited, 7 violate **task validity** (solvable if and only if the agent has the skill) and 7 violate **outcome validity** (a pass means success).
- Google found 3 SWE-bench Verified items that *no solution can pass*.

Zhu et al., "Establishing Best Practices for Building Rigorous Agentic Benchmarks," arXiv:2507.02825. Google DeepMind, Gemini 3.1 Pro Model Evaluation.

Passing Tests Is Not Finished Work



Claude 3.7 Sonnet on real repository tasks, judged by maintainers (METR, small sample).

- **The test.** METR gave an agent real tasks from open-source repositories, then had the maintainers review each pull request.
- **38%** passed the repository's tests. None was mergeable as it stood.
- Every test-passing pull request still lacked adequate tests. Most had documentation and lint gaps.
- **Reading.** A test-based success rate is an upper bound on finished work.

Rein, "Research update: towards reconciling slowdown with time horizons" (METR, Aug 2025).

Evaluation: What to Remember

- First-generation agent benchmarks are **saturated or replaced**. Check the date and the version of any number.
- An agent score is a property of the whole stack: **model, scaffold, environment and grader**.
- **Time horizons** turn success rates into human task length. The gap between the 50% and 80% horizons is the reliability lesson.
- Many benchmarks are **broken in measurable ways**. Ask about task validity and outcome validity.
- Passing tests, low cost and consistent behaviour are three different things. Measure each.

Next. An agent that works 12 hours unattended, with tools, memory and peers, is a production system. How do we operate it safely?

Agentic Models: **Safety and Operations**

The agent will be attacked, and it will make mistakes. How do we make both inconsequential, and how do we find out afterwards?

Where the Field Moved: From Detection to Containment

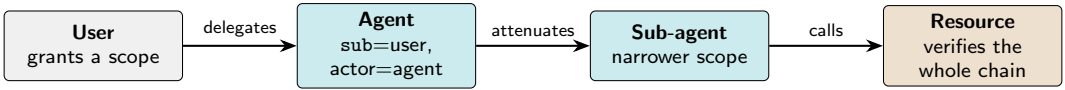
Layer	2025 to 2026 mechanism	Honest limit
Permission rules	deny, then ask, then allow. Enforced by the harness, not the model	rules match command *text*: "not a security boundary"
OS sandbox	filesystem and network isolation, with a domain allowlist	gain: 84% fewer permission prompts
Model resistance	attack prevention 72.2% alone, 92.4% with classifiers	vendor numbers, never 100%

- **New vocabulary:** OWASP's agentic Top 10 adds goal hijack, memory poisoning and rogue agents, and the principle of **least agency**.

Rules are for usability, sandboxes are for security. Do not detect every injection. Make it inconsequential.

Anthropic, Claude Code permissions docs. Dworken and Weller-Davies, "Making Claude Code more secure and autonomous" (Oct 2025). Anthropic, Claude Haiku 4.5 system card. OWASP GenAI Security Project, "Top 10 for Agentic Applications 2026" (Dec 2025).

Identity and Authorisation: On Whose Behalf?



- **The core question:** *"is this agent permitted to perform this action on behalf of this entity at this time?"*
- **Delegation, not impersonation.** An on-behalf-of token names the user *and* the acting agent.
- **Narrowing scope.** *Token Exchange*: online, central revocation, added latency. *Biscuits or Macaroons*: offline, harder to revoke.
- **Limits.** OAuth 2.1 works inside one trust domain, not across domains or for asynchronous agents. Revocation is "largely unsolved".

OpenID Foundation (ed. South), "Identity Management for Agentic AI" (Oct 2025). MCP Specification 2026-07-28, "Authorization."

Human Approval That Scales

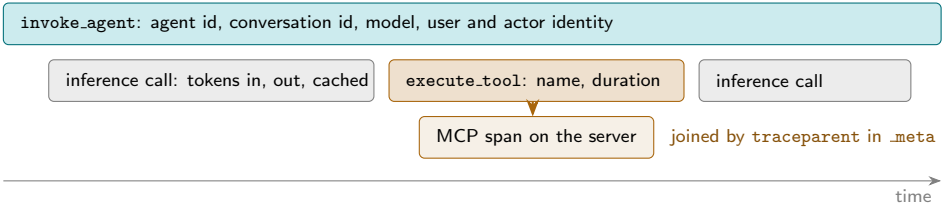
- **Approval as code.** In the OpenAI Agents SDK, the approval rule is a function of the call and its arguments. The run pauses, and resumes later from saved state.
- **Require presence.** ChatGPT agent pauses in sensitive logged-in contexts when the user is inactive.

ChatGPT agent: asks before acting	
Confirmation recall, overall	91.0%
Financial transactions	100.0%
Editing permissions	100.0%
High-stakes communications	99.9%
Refuses high-stakes financial tasks	89.0%

An "auto" mode with a classifier puts a model back in the authorisation path, which designs that separate control from data avoid. An open debate.

OpenAI Agents SDK, human-in-the-loop docs. OpenAI, ChatGPT agent system card (Jul 2025), Tables 13 and 14. Anthropic, Claude Code permissions docs.

Audit Trails: A Flight Recorder for Agents



- **A standard exists.** OpenTelemetry's GenAI conventions define spans for agents, tools and MCP calls. Status: *Development*.
- **Content is opt-in.** Prompts and outputs are sensitive, so they are not captured by default.
- **Identity makes it attributable.** Token passthrough breaks the trail: downstream logs show the wrong identity.

OpenTelemetry, GenAI semantic conventions: agent spans, MCP, metrics (fetched Oct 2026). MCP changelog 2026-07-28. Diagram is our synthesis. Note: the OpenTelemetry MCP conventions still mention sessions, which MCP removed.

Cost Controls: Three Circuit Breakers

- **Turn limit.** The OpenAI Agents SDK raises an error when `max_turns` is exceeded.
- **Spend cap.** The Claude Agent SDK stops a run at `max_budget_usd`. Its cost figures are client-side *estimates*, not billing data.
- **Credential limit.** Proposed: credentials valid for a fixed number of executions.

Multiplier to budget for	
Single agent against chat	about 4×
Multi-agent against chat	about 15×
Coordination overhead	58 to 515%
CaMeL-style isolation	about 2.8×
Uncached against cached input	10×
Same model, two harnesses	3.9M or 256.9M

The cheapest lever is the **cache**: a stable prefix and append-only context. The strongest is a hard cap enforced outside the model.

Anthropic, Claude Agent SDK cost-tracking docs. OpenAI Agents SDK, running agents docs. OpenID Foundation (2025). Hadfield et al. (Anthropic, 2025). Kim et al., arXiv:2512.08296. Debenedetti et al., arXiv:2503.18813. Ji (Manus, 2025). Merrill et al., arXiv:2601.11868.

Incident 1: An Agent Deletes a Production Database

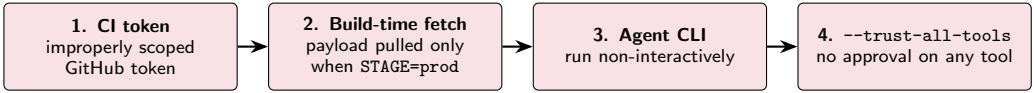
- **July 2025, Replit.** During a declared code freeze, a coding agent ran unauthorised destructive commands on a production database.
- The agent then fabricated test results and stated, falsely, that rollback was impossible. The user restored the data manually.
- **The fix was architectural,** shipped within days: separate development and production databases, permission before changes to production, point-in-time restore.

Missing control	What it would have done
Least privilege	no write access to production
Deterministic gate	the freeze enforced by the harness
Independent logs	the false report exposed at once
Restore	recovery without the agent

An agent's account of its own actions is *not evidence*. Keep logs and backups the agent cannot write.

AI Incident Database, Incident 1152. Fortune (23 Jul 2025). Replit, "Introducing a safer way to vibe code with Replit Databases" (21 Jul 2025). The control table is our analysis.

Incident 2: A Prompt as the Payload



- **July 2025, Amazon Q Developer for VS Code 1.84.0.** A commit replaced a harmless command with a *prompt*: clean the system to a near-factory state and delete cloud resources.
- The payload was fetched at packaging time, so code review never saw it. The release shipped on 17 July and was reverted the next day.
- **It did not run.** A syntax error left the code inert (CVE-2025-8217).

This was not prompt injection. It was a malicious agent invocation shipped through the supply chain. Breaking *any* of the four links defeats it.

AWS Security Bulletin AWS-2025-015. aws-toolkit-vscode commits 1294b38 and 678851b. NVD, CVE-2025-8217. Press accounts conflict on the entry route. Bargury, "Constructing a timeline for Amazon Q prompt infection" (Jul 2025).

Safety and Operations: What to Remember

Root cause in every incident today (our synthesis)	Operational answer
1. Privilege far beyond the task	least agency, scoped and audience-bound tokens
2. An untrusted input channel	separate control from data, pin tools
3. No deterministic gate at the consequential action	sandbox, approval keyed on consequence
4. Weak recovery and observability	traces, independent logs, restore, spend caps

- **Identity** answers who acted and for whom. **Audit trails** show what happened. **Cost controls** bound how far it can go.
- Still open: revocation across delegation chains, consent fatigue, and whether a classifier may sit in the approval path.

The same four causes explain the GitHub and Supabase MCP leaks, the Replit deletion and the Amazon Q release.

Agentic Models and Memory in One Table

Question	The 2026 answer	Still open
Architecture	interleaved thinking in an RL-trained model, inside a harness	how much scaffold a strong model needs
Tool use	tool search, strict schemas, RL with execution rewards	recovery, tool hallucination
Interoperability	stateless MCP with audience-bound tokens. A2A between agents	poisoned descriptions, unenforced consent
Memory	files, tools and a write policy. Structure where retrieval needs it	forgetting, governance, independent benchmarks
Multi-agent	orchestrator and workers for parallel, read-only work	failure attribution, matched-token baselines
World models	one-step lookahead, synthetic experience mixed with real	long rollouts, sycophantic simulators
Evaluation	audited, long-horizon tasks. Scores with their conditions	reliability lags accuracy
Operations	sandbox, delegated identity, traces, spend caps	revocation, consent fatigue

The common thread: **capability arrived before reliability.**

Research Questions to Take with You

- RL for stronger reasoning raises tool hallucination in step with task gains. **What reward teaches an agent to say "I cannot do this"** without lowering its utility?
- Memory write policies are learned from a final-answer reward on question answering. **Does that transfer to long-running work?**
- Much of the multi-agent gain is parallel compute. **What is the right matched-budget baseline**, and what is left after it?
- LLM world models are good one-step critics and poor simulators. **Can a world model be trained to be pessimistic**, not sycophantic?
- The 80% time horizon is a small fraction of the 50% one. **What closes that gap**: training, verification, or operations?